

Relatório de avaliação

Data de geração: 24/11/2025

Resumo da Avaliação

- Total de Questões: 6
- Acertos: 5
- Erros: 1
- Taxa de Acerto: 83%

Material de Estudo

TEMA 4 - CONVERSÕES ENTRE COLLECTIONS

As diferentes coleções do C# permitem que você utilize métodos para conversão entre uma e outra, a depender do tipo. Isso pode ser muito útil quando você precisa manter os elementos já criados em uma coleção, mas mudar a estrutura de dados dessa coleção para outro tipo. Certamente você pode imaginar que, como todas as coleções implementam `IEnumerable<T>`, basta enumerar seus elementos e adicionar um novo Type de coleção (como por exemplo, enumerar um Hashset e depois adicionar seus valores em uma Lista). Apesar dessa técnica funcionar, ela certamente será muito mais lenta que os métodos que veremos a seguir.

Para entender a base das conversões entre coleções, e porque o processo é eficiente em termos de alocação de memória e performance, precisamos retomar o assunto do início da aula: Arrays. Todas as coleções, invariavelmente, armazenam os dados em memória, utilizando-se de Arrays, o que significa que ele é a base comum para todas as coleções e sequências de dados no C#. Como já vimos, os Arrays apresentam um conceito de implementação e representação nativo na CLR, o que faz com que sejam ideais para isso (Griffiths, 2019, p. 230).

Vamos criar um exemplo em que teremos uma classe Pessoa, contendo uma propriedade Nome (string) e Idade (int). Depois, vamos criar um `Pessoa[]` (array de Pessoa), uma `List<Pessoa>`, um `HashSet<Pessoa>` e um `Dictionary<int, Pessoa>`. Nesse exemplo, vamos primeiro popular nosso Array com 5 milhões de registros. Depois, vamos observar alguns padrões de

conversão entre Arrays e Listas. Na sequência, vamos estudar o método “CopyTo”.

Observe que fizemos o override do método ToString() (método comum a todos os objetos no C#). Com a sobrescrita do método ToString, vamos escrever no console o Nome e a Idade dessa “Pessoa”.

Vamos inicializar os Arrays e Listas e criar 5 milhões de objetos Pessoa na memória:

Agora que temos um Array com 5 milhões de objetos, vamos fazer a nossa primeira “conversão”. Suponha que você tem um Array de objetos (podem ser int, long, DateTime ou Object – qualquer objeto). Você deseja copiar o conteúdo deste Array para uma List<T>. Uma boa solução é aproveitar o método “AddRange” da List<T>. Esse método copia o conteúdo do Array para a Lista. Uma observação importante: o conteúdo do Array é apenas uma referência ao objeto real.

Observe que HashSets não permitem adicionar valores em massa, como a Lista. Além disso, a estrutura de um HashSet depende de um conceito matemático chamado Hashtable. Porém, o HashSet nos provê um modo de inicializar um HashSet<T> utilizando uma ICollection<T>. Internamente, ele fará o mesmo que a lista, copiando a referência de array e gerando um Hashtable interno. No exemplo a seguir, vemos que o HashSet mantém a referência para o objeto, mesmo com uma estrutura de Array internamente diferente:

podemos ver como a estrutura HashSet mantém os seus dados na memória. Ela utiliza basicamente dois arrays internos: um para armazenar um índice do objeto e outro para armazenar a referência. O segundo Array também guarda o valor do índice dentro de uma struct similar a do Dictionary (KeyValuePair). Apesar de parecer estranho, esse conceito está fundamentado no princípio de gerar elevada performance na busca, através de uma função de dispersão e de uma estrutura de índices.

a estrutura de índices nem sempre aponta para um valor.

Isso ocorre porque a função de dispersão, que é executada sempre que se adiciona um elemento em um HashSet, gera um índice para esse elemento com base em um procedimento matemático, e não em uma sequência, como no caso de Listas e Arrays. Esse é o motivo pelo qual não se pode buscar um elemento no HashSet com base em seu índice. Afinal, esse índice é controlado internamente pela coleção, e ela não ocupa todas as suas posições de forma

linear, e sim de forma dispersa (Livro C# 8.0).

Inicializar um Dictionary<TKey, TValue> é um pouco mais complicado.

Enquanto o HashSet gera índices internos para seus objetos, o Dictionary obriga você a fornecer a chave do “índice”, passando um struct KeyValuePair como parâmetro para cada inserção na coleção. Vamos fazer isso enumerando a nossa lista e inserindo-a no dicionário. Na sequência, veremos como fazer isso usando funções de extensão que facilitam muito a nossa vida no dia a dia do C#.

Perguntas e Respostas

1. Como podemos copiar o conteúdo de um Array para uma List<T> no C#?

- Resposta do usuário: Gerar uma estrutura de índices e copiar a referência
 - Correto: Sim
-

2. Qual é a função do segundo Array internamente utilizado pelo HashSet?

- Resposta do usuário: Guardar o valor do índice dentro de uma struct similar a do Dictionary
 - Correto: Sim
-

3. Qual conceito base comum para todas as coleções e sequências de dados no C#?

- Resposta do usuário: capacidade de serem iteradas
 - Correto: Não
 - Feedback: "Incorreto. A resposta certa é: Implementação e representação nativa na CLR"
-

4. Qual função matemática é executada sempre que se adiciona um elemento em um HashSet?

- Resposta do usuário: função de hash
 - Correto: Sim
-

5. Podemos buscar um elemento no HashSet com base em seu índice?

- Resposta do usuário: não
 - Correto: Sim
-

6. Quando inicializar um Dictionary<TKey, TValue>, você precisa fornecer a chave

do 'índice'?

- Resposta do usuário: sim
- Correto: Sim

Documento gerado automaticamente pelo sistema de avaliação perguntAI.